

⚡ Web Performance & Optimization — গতি, স্কেলাবিলিটি ও ইউজার সন্তুষ্টির মূল ভিত্তি

Web application শুধু কাজ করলেই আধুনিক ইঞ্জিনিয়ারিংয়ের দায়িত্ব শেষ হয় না—এটি যেন **বিদ্যুৎ গতিতে লোড হয়, চোখের পলকে রেসপন্ড করে, ব্যাটারি ও মেমোরি সাশ্রয়ী হয় এবং মসৃণ ইউজার এক্সপেরিয়েন্স দেয়**, সেটাই চূড়ান্ত লক্ষ্য। এই চ্যাপ্টারে আমরা **Critical Rendering Path, Layout Thrashing, Debounce/Throttle, Lazy Loading, Code Splitting, Caching & CDN, Web Workers, Core Web Vitals (LCP, INP, CLS)** এবং প্রোডাকশন অপটিমাইজেশন পদ্ধতি শিখব।

- ⚡ 1. CRP & DOM Flow
- ⚡ 2. Layout Thrashing
- ⚡ 3. Bundle & Tree Shake
- ⚡ 4. Modern Image Formats
- ⚡ 5. Dynamic Code Split
- ⚡ 6. Debounce & Throttle
- ⚡ 7. 60 FPS & rAF
- ⚡ 8. HTTP Cache & CDN
- ⚡ 9. Workers & Memory
- ⚡ 10. Core Web Vitals

Part 01 — Foundations of Web Performance & Page Load Lifecycle (33.1 – 33.5)

33.1 Web Performance কী?

Web Performance হলো একটি website/web application কত দ্রুত এবং efficiently user-এর জন্য কাজ করে তার সামগ্রিক পরিমাপ। সহজভাবে:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
User
↓
Website Open
↓
HTML Load
↓
CSS Load
↓
JavaScript Load
↓
Content Render
↓
User Interaction
```

এই পুরো process যত efficient হবে, website তত performant হবে।

33.2 কেন Performance গুরুত্বপূর্ণ?

ধরো দুইটি website:

♦ Website A

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Click
↓
5 seconds wait
↓
Result
```

♦ Website B

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Click
↓
Fast response
↓
Result
```

User সাধারণত দ্রুত feedback পাওয়া application-কে বেশি usable মনে করবে। Performance-এর প্রভাব পড়ে:

- User Experience
- Accessibility
- Conversion
- SEO
- Mobile usability
- Server cost
- Battery usage

33.3 Performance-এর প্রধান তিনটি অংশ

একটি website performance broadly তিন ভাগে ভাবতে পারো:

⚡ NETWORK WATERFALL & CACHING PIPELINE

Web Performance



33.4 Performance Bottleneck কী?

Bottleneck হলো এমন অংশ যেটি পুরো system-এর performance আটকে দিচ্ছে। Example:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
HTML      → 100ms
CSS        → 200ms
JS         → 5 seconds
Image      → 300ms
```

এখানে JavaScript বড় bottleneck হতে পারে।

33.5 Page Load-এর Basic Flow

Browser সাধারণত:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
URL
↓
DNS
↓
Connection
↓
HTTP Request
↓
Server Response
↓
HTML
↓
Parse HTML
↓
CSS/JS/Images
↓
Render
↓
Interaction
```

এখানে যেকোনো জায়গায় delay হতে পারে।

Part 02 — Critical Rendering Path, Layout, Paint & Layout Thashing (33.6 – 33.15)

33.6 Critical Rendering Path

Browser-এর page render করার গুরুত্বপূর্ণ process:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
HTML
↓
DOM
```

⚡ CRITICAL RENDERING PATH & BROWSER PIPELINE

```
CSS
↓
CSSOM
```

তারপর:

⚡ CRITICAL RENDERING PATH & BROWSER PIPELINE

```
DOM + CSSOM
↓
Render Tree
↓
Layout
↓
Paint
↓
Composite
```

33.7 DOM

HTML থেকে browser তৈরি করে:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

HTML
↓
DOM

Example:

HTML DOM MARKUP

HTML

```
<h1>Hello</h1>
<p>World</p>
```

Conceptually:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Document
├── h1
│   └── "Hello"
└── p
    └── "World"
```

33.8 CSSOM

CSS থেকে browser তৈরি করে:

⚡ CRITICAL RENDERING PATH & BROWSER PIPELINE

CSS
↓
CSSOM

তারপর:

⚡ CRITICAL RENDERING PATH & BROWSER PIPELINE

DOM + CSSOM
↓
Render Tree

33.9 Layout

Browser calculate করে:

- width
- height
- position
- spacing

এটি layout/reflow process-এর অংশ। Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
element.style.width = "500px";
```

এতে layout recalculation প্রয়োজন হতে পারে।

33.10 Paint

Browser pixels আঁকে:

⚡ LAYOUT THRASHING VS BATCHED EXECUTION FLOW

Layout

↓

Paint

↓

Pixels

যেমন:

- text
- background
- border
- shadow

33.11 Composite

Browser বিভিন্ন layer combine করে final screen তৈরি করতে পারে।

⚡ CRITICAL RENDERING PATH & BROWSER PIPELINE

Painted Layers

↓

Composite

↓

Screen

33.12 Reflow / Layout

DOM বা CSS-এর কোনো change-এর কারণে layout আবার calculate করতে হলে তাকে সাধারণভাবে **reflow/layout recalculation** বলা হয়। Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
box.style.width = "500px";  
box.style.height = "300px";
```

অনেক layout-affecting changes করলে browser-এর কাজ বাড়তে পারে।

33.13 Repaint

কোনো visual property change হলে repaint হতে পারে। যেমন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
box.style.backgroundColor = "red";
```

সব property একইভাবে layout trigger করে না।

33.14 Layout Thashing

এটি একটি common performance problem। ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const box = document.querySelector(".box");

for (let i = 0; i < 1000; i++) {
  box.style.width = i + "px";

  console.log(box.offsetWidth);
}
```

এখানে বারবার:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Write
↓
Read layout
↓
Write
↓
Read layout
```

করার ফলে browser-কে repeated layout calculation করতে হতে পারে।

33.15 Better Approach

আগে values calculate:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const widths = [];

for (let i = 0; i < 1000; i++) {
  widths.push(i);
}

for (const width of widths) {
  box.style.width = width + "px";
}
```

আরও ভালো architecture/context অনুযায়ী batching বা CSS class ব্যবহার করা যায়।

Part 03 — DOM Batching, Bundle Size, Tree Shaking & Compression (33.16 – 33.22)

33.16 DOM Manipulation Optimize করা

✗ বারবার DOM change:

JAVASCRIPT SOURCE

JAVASCRIPT

```
for (let i = 0; i < 1000; i++) {
  const p = document.createElement("p");
  p.textContent = `Item ${i}`;

  document.body.appendChild(p);
}
```

এটি সবসময় catastrophic নয়, কিন্তু বড় DOM-এর ক্ষেত্রে repeated mutation expensive হতে পারে।

33.17 DocumentFragment

একসাথে DOM তৈরি করে শেষে append করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const fragment = document.createDocumentFragment();

for (let i = 0; i < 1000; i++) {

  const p = document.createElement("p");

  p.textContent = `Item ${i}`;

  fragment.appendChild(p);
}

document.body.appendChild(fragment);
```

Concept:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Create many elements
      ↓
Fragment
      ↓
One DOM insertion
```

33.18 JavaScript Bundle Size

ধরো:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
app.js = 5 MB
```

Browser-কে এটি download, parse এবং execute করতে হবে। অন্যদিকে:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
app.js = 200 KB
```

সাধারণত কম data transfer ও processing দরকার হবে। তাই:

💡 **Smaller JavaScript is generally better.**

33.19 Tree Shaking

Unused code বাদ দেওয়ার build optimization। Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
export function add() {}
export function subtract() {}
export function multiply() {}
```

তুমি:

JAVASCRIPT SOURCE

JAVASCRIPT

```
import { add } from './math.js';
```

শুধু add() ব্যবহার করলে build tool unused export বাদ দেওয়ার সুযোগ পেতে পারে।

33.20 Minification

Source code ছোট করা। Before:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function calculateTotal(price, quantity) {  
  return price * quantity;  
}
```

Minified concept:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function calculateTotal(a,b){return a*b}
```

ফলে file size কমতে পারে।

33.21 Compression

Server/client data transfer-এর সময় compression ব্যবহার করা যায়। Common compression:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

Gzip
Brotli

Concept:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Original File  
↓  
Compression  
↓  
Smaller Transfer  
↓  
Browser  
↓  
Decompression
```

33.22 Brotli

Modern web delivery-তে Brotli (br) compression commonly ব্যবহৃত হয়। বিশেষ করে text assets-এর জন্য:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

HTML
CSS
JS
JSON

compression size কমাতে সাহায্য করতে পারে।

Part 04 — Media & Image Optimization, Responsive & Lazy Loading (33.23 – 33.27)

33.23 Image Optimization

Website slow হওয়ার বড় কারণ image হতে পারে। ধরো:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
hero-image.png  
= 8 MB
```

এটি unnecessary বড়। Optimize করা যায়:

- resize
- compression
- responsive images
- modern formats
- lazy loading

33.24 Modern Image Formats

Common formats:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

JPEG
PNG
WebP
AVIF
SVG

যেমন photographic image-এর ক্ষেত্রে WebP/AVIF অনেক ক্ষেত্রে ছোট হতে পারে।

33.25 Responsive Images

HTML DOM MARKUP

HTML

```

```

Browser ধোঁয়াজন অনুযায়ী appropriate image resource নির্বাচন করতে পারে।

33.26 Lazy Loading

যে resource এখনই দরকার নেই, সেটি পরে load করা। Image:

HTML DOM MARKUP

HTML

```

```

JavaScript-এর ক্ষেত্রেও code splitting/lazy import ব্যবহার করা যায়।

33.27 Eager vs Lazy

♦ Eager

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

Page Load



Everything load

♦ Lazy

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

Page Load



Only necessary content



User scroll/click



Additional content load

33.28 JavaScript Dynamic Import

ES Modules-এ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const module = await import("./heavy-module.js");
```

এটি code-এর একটি অংশ প্রয়োজন হলে load করতে সাহায্য করতে পারে।

33.29 Code Splitting

Large JavaScript application ভাগ করা:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
app.js
↓
chunk-1.js
chunk-2.js
chunk-3.js
```

User যে feature ব্যবহার করবে তার প্রয়োজনীয় chunk load করা যেতে পারে। React application-এ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const Dashboard = lazy(
  () => import("./Dashboard.jsx")
);
```

Conceptually:

⚡ DYNAMIC CODE SPLITTING & ROUTE LAZY LOADING

```
Initial Bundle
↓
Small
↓
User opens Dashboard
↓
Dashboard chunk loads
```

33.30 Debouncing

Rapid events-এর মধ্যে user থামার পর function execute করা। ধরো search input:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

User types:

H
He
Hel
Hell
Hello

প্রতিটি keystroke-এ API call করলে:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

5 keystrokes
↓
5 API requests

Debounce:

⚡ DEBOUNCE VS THROTTLE EXECUTION TIMELINE

User typing
↓
Wait
↓
User stops
↓
One API request

⚡ Debounce implementation

JAVASCRIPT SOURCE

JAVASCRIPT

```
function debounce(fn, delay) {

  let timer;

  return function (...args) {

    clearTimeout(timer);

    timer = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}
```

Use:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const search = debounce((value) => {
  console.log("Searching:", value);
}, 500);
```

33.31 Throttling

Throttle নির্দিষ্ট সময়ের মধ্যে function execution limit করে। ধরো scroll event:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Scroll
Scroll
Scroll
Scroll
Scroll
Scroll
Scroll
```

Throttle করলে:

⚡ DEBOUNCE VS THROTTLE EXECUTION TIMELINE

```
Execute
↓
Wait
↓
Execute
↓
Wait
↓
Execute
```

⚡ Throttle example

JAVASCRIPT SOURCE

JAVASCRIPT

```
function throttle(fn, delay) {

  let lastTime = 0;

  return function (...args) {

    const now = Date.now();

    if (now - lastTime ≥ delay) {

      lastTime = now;

      fn.apply(this, args);

    }

  };
}
```

33.32 Debounce vs Throttle

Debounce	Throttle
User থামলে execute	নির্দিষ্ট interval-এ execute
Search	Scroll
Resize	Mouse move
Typing	Drag

💡 মনে রাখো:

⚡ DEBOUNCE VS THROTTLE EXECUTION TIMELINE

```
Debounce → "Wait until quiet"
Throttle → "Limit frequency"
```

33.33 Event Delegation

অনেক child element-এ আলাদা listener না দিয়ে parent-এ listener দেওয়া যায়। HTML:

HTML DOM MARKUP

HTML

```
<ul id="list">
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
</ul>
```

JavaScript:

JAVASCRIPT SOURCE

JAVASCRIPT

```
document
  .querySelector("#list")
  .addEventListener("click", (event) => {

    if (event.target.matches("li")) {
      console.log(event.target.textContent);
    }
  });
```

এতে অনেক listener না দিয়েও কাজ করা যায়।

33.34 Passive Event Listener

Touch/scroll event-এ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
window.addEventListener(
  "scroll",
  handleScroll,
  { passive: true }
);
```

passive: true browser-কে জানায় listener preventDefault() ব্যবহার করবে না। কিছু scrolling scenario-তে এটি performance/scroll responsiveness-এর জন্য সাহায্য করতে পারে।

Part 07 — Rendering Performance, rAF & GPU Acceleration (33.35 – 33.39)

33.35 requestAnimationFrame()

Animation বা visual update-এর জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
requestAnimationFrame(() => {
  box.style.transform = "translateX(100px)";
});
```

Browser-এর rendering cycle-এর সাথে visual updates synchronize করতে এটি usefull

33.36 কেন setInterval() সব animation-এর জন্য ideal নয়?

এভাবে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
setInterval(() => {
  box.style.left = "...";
}, 16);
```

করার বদলে animation-এর জন্য সাধারণত:

JAVASCRIPT SOURCE

JAVASCRIPT

```
requestAnimationFrame()
```

better choice হতে পারে।

33.37 Animation Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
let position = 0;

function animate() {

  position += 2;

  box.style.transform =
    `translateX(${position}px)`;

  if (position < 500) {
    requestAnimationFrame(animate);
  }
}

requestAnimationFrame(animate);
```

33.38 CSS Transform Performance

Animation-এর ক্ষেত্রে অনেক সময়:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
transform: translateX( ... );
```

এবং:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
opacity: ...
```

এর মতো properties browser rendering pipeline-এর জন্য ভালোভাবে optimize করা যায়। অন্যদিকে frequent:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
top
left
width
height
```

change করলে layout work হতে পারে। এটি absolute rule নয়—browser implementation এবং surrounding layout-এর ওপরও নির্ভর করে।

33.39 will-change

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
.box {
  will-change: transform;
}
```

Browser-কে future change সম্পর্কে hint দিতে পারে। ⚠️ সব element-এ:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
will-change: transform;
```

দেওয়া উচিত নয়। কারণ excessive layer/resource usage হতে পারে।

33.40 Browser Caching

Browser static resource cache করতে পারে। যেমন:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
logo.png
style.css
app.js
```

প্রথমবার download:

⚡ NETWORK WATERFALL & CACHING PIPELINE

```
Server
↓
Browser
↓
Cache
```

পরেরবার cache policy অনুযায়ী network request কমতে পারে।

33.41 HTTP Cache Headers

Important concepts:

⚡ NETWORK WATERFALL & CACHING PIPELINE

```
Cache-Control
ETag
Last-Modified
Expires
```

Example:

HTTP CACHE HEADERS

HTTP

```
Cache-Control: public, max-age=31536000
```

মানে resource cache করার policy নির্দেশ করা হচ্ছে।

33.42 Cache-Control

Common concepts:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
max-age
no-cache
no-store
public
private
```

◆ no-store

Response store না করার নির্দেশ দেয়। Sensitive data-এর ক্ষেত্রে গুরুত্বপূর্ণ হতে পারে।

◆ max-age

কত সময় fresh হিসেবে treat করা হবে তার নির্দেশ দিতে পারে।

33.43 ETag

Server resource-এর একটি identifier দিতে পারে:

HTTP CACHE HEADERS

HTTP

ETag: "abc123"

পরবর্তী request-এ browser পাঠাতে পারে:

HTTP CACHE HEADERS

HTTP

If-None-Match: "abc123"

Resource পরিবর্তন না হলে server:

HTTP CACHE HEADERS

HTTP

304 Not Modified

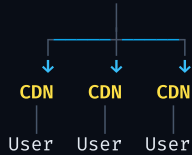
দিতে পারে।

33.44 CDN

CDN = Content Delivery Network Static assets user-এর কাছাকাছি edge location থেকে serve করতে পারে।

⚡ NETWORK WATERFALL & CACHING PIPELINE

Origin Server



Benefits:

- lower latency
- distributed delivery
- caching
- reduced origin load

33.45 Network Waterfall

Browser DevTools-এর Network tab-এ দেখতে পারো:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

HTML

- CSS
- JS
- Image
- Font
- API

প্রতিটির:

- request time
- response time
- size
- status
- initiator

দেখা যায়।

33.46 Chrome DevTools Performance

Browser DevTools → **Performance** এখানে recording করে দেখতে পারো:

⚡ LAYOUT THRASHING VS BATCHED EXECUTION FLOW

```
JavaScript execution
Rendering
Painting
Layout
Long Tasks
Frames
```

33.47 Long Task

Main thread-এ খুব দীর্ঘ JavaScript task হলে UI responsiveness কমে যেতে পারে। Concept:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
User Click
↓
Heavy JS
↓
Main Thread Busy
↓
UI delayed
```

এজন্য heavy work:

- break into smaller chunks
- Web Worker
- lazy loading
- optimized algorithms

ইত্যাদি দিয়ে handle করা যায়।

33.48 Main Thread

Browser-এর main thread সাধারণত:

⚡ LAYOUT THRASHING VS BATCHED EXECUTION FLOW

```
JavaScript
+
DOM
+
Style/Layout related work
+
User Interaction
```

এর বড় অংশ handle করে। যদি JavaScript:

JAVASCRIPT SOURCE

JAVASCRIPT

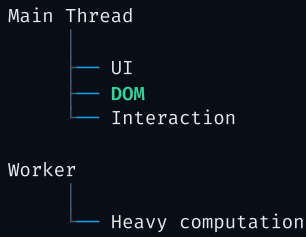
```
while (true) {
  // never ending
}
```

চালাও, UI freeze হয়ে যেতে পারে।

33.49 Web Worker আবার কেন গুরুত্বপূর্ণ?

Chapter 32-এ আমরা Web Worker দেখেছি। Heavy computation:

⚡ MAIN THREAD VS WEB WORKER CONCURRENCY



এতে main thread-এর ওপর চাপ কমানো যায়।

33.50 Memory Performance

Performance শুধু speed নয়। Memory usage-ও গুরুত্বপূর্ণ। Potential সমস্যা:

⚡ NETWORK WATERFALL & CACHING PIPELINE

Huge Array
Huge Object
Unremoved Event Listener
Detached DOM
Growing Cache

এগুলো memory usage বাড়াতে পারে। Chapter 30-এর Garbage Collection knowledge এখানে কাজে লাগে।

33.51 Memory Leak Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
const cache = [];  
  
function addData(data) {  
  cache.push(data);  
}
```

যদি cache কখনো clean না হয়:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
cache  
↓  
Data  
↓  
More Data  
↓  
More Data  
↓  
More Data
```

Memory usage ক্রমেই বাড়তে পারে।

33.52 Image Lazy Loading + IntersectionObserver

Chapter 32-এর knowledge combine করলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const images =
  document.querySelectorAll("img[data-src]");

const observer =
  new IntersectionObserver(entries => {

    entries.forEach(entry => {

      if (!entry.isIntersecting) return;

      const img = entry.target;

      img.src = img.dataset.src;

      observer.unobserve(img);
    });
  });

images.forEach(img => observer.observe(img));
```

এটি একটি real performance technique।

33.53 Code Splitting + Dynamic Import

JAVASCRIPT SOURCE

JAVASCRIPT

```
button.addEventListener("click", async () => {

  const module =
    await import("./heavyFeature.js");

  module.start();
});
```

User click না করা পর্যন্ত heavy feature-এর code load না করানো যেতে পারে।

33.54 Preload

কিছু critical resource আগে load করার জন্য:

HTML DOM MARKUP

HTML

```
<link
  rel="preload"
  href="/fonts/main.woff2"
  as="font"
  type="font/woff2"
  crossorigin
>
```

⚠️ Preload শুধু genuinely critical resource-এর জন্য ব্যবহার করা ভালো।

33.55 Prefetch

Future navigation-এর সম্ভাব্য resource আগে fetch করতে:

HTML DOM MARKUP

HTML

```
<link
  rel="prefetch"
  href="/next-page.js"
>
```

Browser support/priority behavior অনুযায়ী actual loading timing পরিবর্তিত হতে পারে।

33.56 defer vs async

HTML:

HTML DOM MARKUP

HTML

```
<script src="app.js" defer></script>
```

♦ defer

HTML parsing block না করে script download করতে পারে এবং document parse শেষ হলে execution করে। Multiple deferred scripts সাধারণত document order অনুযায়ী execute হয়।

♦ async

HTML DOM MARKUP

HTML

```
<script src="analytics.js" async></script>
```

Download complete হলে execution শুরু হতে পারে, parsing-এর সাথে timing overlap করতে পারে। একাধিক async script-এর execution order guaranteed নয়।

33.57 Comparison

defer	async
Download parallel হতে পারে	Download parallel হতে পারে
HTML parsing block না করে	HTML parsing block না করে
Parse complete হওয়ার পরে execute	Download শেষ হলেই execute হতে পারে
Order maintained	Order guaranteed নয়

33.58 Critical CSS

Above-the-fold content-এর জন্য প্রয়োজনীয় CSS দ্রুত দিতে critical CSS strategy ব্যবহার করা যায়। Concept:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Critical CSS
↓
Initial render
↓
Remaining CSS
↓
Later
```

33.59 Font Performance

Web fonts performance affect করতে পারে। Strategies:

- WOFF2 ব্যবহার
- প্রয়োজনীয় font weights কমানো
- preload carefully
- font-display ব্যবহার

Example:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
@font-face {  
  font-family: "MyFont";  
  src: url("/font.woff2") format("woff2");  
  font-display: swap;  
}
```

33.60 API Performance

Frontend slow হওয়ার কারণ সবসময় frontend নয়। ধরো:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Button Click  
↓  
API Request  
↓  
Server  
↓  
Database  
↓  
Server  
↓  
Response  
↓  
UI
```

Database slow হলে frontend-ও slow মনে হবে।

33.61 API Optimization

Backend side:

- pagination
- caching
- database indexes
- efficient queries
- compression
- response shaping
- connection pooling

Frontend side:

- debounce search
- cache responses where appropriate
- avoid duplicate requests
- pagination/infinite loading
- abort stale requests

33.62 AbortController

Unnecessary request cancel করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const controller = new AbortController();

fetch("/api/search", {
  signal: controller.signal
});
```

Cancel:

JAVASCRIPT SOURCE

JAVASCRIPT

```
controller.abort();
```

♦ Real-life

Search:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

User types:

Java



Request 1

User types:

JavaScript



Request 2

আগের unnecessary request cancel করা যেতে পারে।

Part 11 — Core Web Vitals, Performance Budget & Production Checklist (33.63 – 33.70)

33.63 Performance Measurement

Optimization করার আগে measure করতে হবে। Golden rule:

🔦 Measure → Identify → Optimize → Measure Again

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

Measure



Find **Bottleneck**



Optimize



Measure Again

শুধু অনুমান করে optimization করা উচিত নয়।

33.64 Core Web Vitals

Modern web performance-এর গুরুত্বপূর্ণ user-centric metrics:

♦ LCP

Largest Contentful Paint Main content কত দ্রুত visible হয় তার একটি গুরুত্বপূর্ণ metric।

♦ INP

Interaction to Next Paint User interaction-এর পরে page কত দ্রুত visual response দেয় তার metric।

♦ CLS

Cumulative Layout Shift Page-এর content unexpected ভাবে কতটা নড়ে যায় তা measure করে।

33.65 LCP

Example:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Page open
↓
Hero heading/image
↓
Main content visible
```

এই গুরুত্বপূর্ণ content দ্রুত render হওয়া ভালো। LCP improve করতে:

- optimize images
- reduce server delay
- preload critical resources carefully
- reduce render-blocking resources
- optimize HTML/CSS/JS

33.66 INP

ধরো:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
User clicks button
↓
Heavy JavaScript
↓
UI responds late
```

এটি poor interaction responsiveness-এর sign। Improve করতে:

- long tasks break করা
- unnecessary JS কমানো
- event handlers optimize
- Web Worker
- code splitting

ব্যবহার করা যায়।

33.67 CLS

Example:

⚡ PERFORMANCE LIFECYCLE & EXECUTION FLOW

```
Page load
Text
↓
Image loads
↓
Image জায়গা নেয়
↓
Text নিচে নেমে গেল
```

এটি layout shift।

⚡ CLS improve

Image-এর dimensions define করুন:

HTML DOM MARKUP

HTML

```

```

অথবা CSS aspect ratio reserve করুন।

33.68 loading="lazy"

HTML DOM MARKUP

HTML

```

```

Browser-এর native lazy loading ব্যবহার করা যায়।

33.69 Performance Budget

Project-এর জন্য performance target define করা যায়। Example:

⚡ CORE WEB VITALS METRICS & TARGET BENCHMARKS

```
JavaScript < 300 KB  
Images < 1 MB  
LCP < target  
API response < target
```

এটি team-কে performance-aware development করতে সাহায্য করে।

33.70 Production Optimization Checklist

Deployment-এর আগে:

⚡ NETWORK WATERFALL & CACHING PIPELINE

- ❑ Images optimized
- ❑ Unnecessary JS removed
- ❑ Code splitting considered
- ❑ Lazy loading used where appropriate
- ❑ **Compression** enabled
- ❑ **Cache** strategy configured
- ❑ API optimized
- ❑ No unnecessary API requests
- ❑ Console/debug code reviewed
- ❑ Source maps strategy reviewed
- ❑ Performance measured
- ❑ Mobile tested

Part 12 — Must Know Architecture, Quick Cheat Sheet, 12 Practice Labs & Final Mental Map

33.71 🔥 MUST KNOW — পারফরম্যান্স অপটিমাইজেশনের ১০টি মূল স্তম্ভ

প্রোডাকশন গ্রেড ওয়েব অ্যাপ্লিকেশনের গতি ও রেসপন্সিভনেস নিশ্চিত করার জন্য এই ১০টি আর্কিটেকচারাল প্যাটার্ন সর্বদা মনে রাখা প্রয়োজন:

🚀 1. Rendering Pipeline & DOM

- **DOM + CSSOM:** Render Tree গঠন করে লেআউট তৈরি
- **Layout & Paint:** রিফ্লো কমানো ও GPU কম্পোজিটিং ব্যবহার
- **DocumentFragment:** ১০০০টি নোড মাত্র ১টি রিফ্লোতে ইনসার্ট
- **will-change & CSS Transform:** সিপিইউ থ্রেড বাদ দিয়ে জিপিইউতে এনিমেশন

🚀 2. Runtime Execution & Events

- **Debounce:** সার্চ ইনপুটে টাইপিং থামা পর্যন্ত অপেক্ষা
- **Throttle:** স্ক্রল বা রিসাইজে নির্দিষ্ট ইন্টারভালে এক্সিকিউশন
- **Passive Listeners:** স্ক্রলিংয়ে preventDefault ব্লকিং রোধ
- **Web Workers:** মেইন থ্রেডকে ফ্রি রেখে ব্যাকগ্রাউন্ডে ক্যালকুলেশন

🚀 3. Delivery & Core Web Vitals

- **Code Splitting:** ডাইনামিক import() দিয়ে অন-ডিম্যান্ড মডিউল লোড
- **Brotli & Tree Shaking:** অব্যবহৃত কোড বাদ দিয়ে সাইজ ৭০% হ্রাস
- **Cache-Control & CDN:** ব্যবহারকারীর নিকটবর্তী নোড থেকে ক্যাশ ডেলিভারি
- **Core Web Vitals:** LCP (<2.5s), INP (<200ms), CLS (<0.1) লক্ষ্য পূরণ

Topic	Main Concept & Mechanism
DOM	HTML ডকুমেন্টের ইন-মেমোরি নোড ট্রি গঠন
CSSOM	CSS সিলেক্টর ও স্টাইল রুলসের ক্যাসকেডিং অবজেক্ট ট্রি
Layout (Reflow)	উপাদানের জ্যামিতিক সাইজ ও স্ক্রিনে সঠিক পজিশন গণনা
Paint (Repaint)	পিক্সেল ড্রয়িং (রং, ব্যাকগ্রাউন্ড, শ্যাডো, বর্ডার রেডারিং)
Composite	আলাদা জিপিইউ লেয়ারসমূহকে স্ক্রিনে একত্রিত করা
Debounce	ব্যবহারকারীর অ্যাকশন থামা পর্যন্ত ফাংশন এক্সিকিউশন স্থগিত রাখা
Throttle	নির্দিষ্ট সময় ব্যবধানে সর্বোচ্চ একবার ফাংশন এক্সিকিউট করা
Lazy Loading	উপাদান স্ক্রিনে আসার ঠিক আগ মুহূর্তে লোড করা (loading="lazy")
Code Splitting	বিশাল বান্ডেল ভেঙে ছোট ছোট অন-ডিম্যান্ড চাক্স তৈরি করা
Tree Shaking	বিল্ড টাইমে অব্যবহৃত ডেড-কোড স্বয়ংক্রিয়ভাবে মুছে ফেলা
Minification	হোয়াইটস্পেস ও ভ্যারিয়েবল নাম ছোট করে সাইজ সংকুচিত করা
Compression	ট্রান্সফারের সময় Brotli/Gzip দিয়ে ফাইল সাইজ কমানো
CDN	ভৌগোলিকভাবে ব্যবহারকারীর নিকটবর্তী এজ-সার্ভার থেকে ফাইল পরিবেশন
HTTP Cache	Cache-Control: max-age দিয়ে ব্রাউজারে রিসোর্স পুনব্যবহার
Web Worker	মেইন থ্রেডকে ফ্রি রেখে ব্যাকগ্রাউন্ড থ্রেডে হেভি প্রসেসিং
requestAnimationFrame	ব্রাউজারের রিফ্রেশ রেটের (60FPS/16.6ms) সাথে সিন্ক করে স্মুথ অ্যানিমেশন
AbortController	অপ্রয়োজনীয় বা পুরাতন ফেচ রিকোয়েস্ট তৎক্ষণাৎ বাতিল করা
LCP (Largest Contentful Paint)	পেজের সবচেয়ে বড় কন্টেন্ট রেন্ডার হওয়ার সময় (লক্ষ্য: < 2.5s)
INP (Interaction to Next Paint)	ইউজার ক্লিকের পর ডিজুয়াল রেসপন্স পেতে দেরি (লক্ষ্য: < 200ms)
CLS (Cumulative Layout Shift)	পেজ লোড হওয়ার সময় কন্টেন্টের অনাকাঙ্ক্ষিত নড়াচড়া (লক্ষ্য: < 0.1)

Lab #1 [BEGINNER] — Debounced Search Input with Live Console Logging

সমস্যা ও লক্ষ্য: একটি search input তৈরি করুন এবং debounce ব্যবহার করে console-এ search value print করুন। টাইপিং থামার ৫০০ms পর রিকোয়েস্ট কার্যকর হবে।

HTML DOM STRUCTURE

HTML

```
<!-- HTML Search Input Markup -->
<div class="search-box">
  <input type="text" id="search-input" placeholder="Search products, documentation ..." autocomplete="off">
  <div id="search-status">Typing triggers debounced API call after 500ms idle</div>
</div>
```

SOLUTION IMPLEMENTATION (LAB #1)

JAVASCRIPT

```
// 1. Debounce Utility Function
function debounce(func, delay = 500) {
  let timerId = null;
  return function(...args) {
    clearTimeout(timerId); // পূর্বের টাইমার বাতিল
    timerId = setTimeout(() => {
      func.apply(this, args); // বিলম্ব শেষে মূল ফাংশন কল
    }, delay);
  };
}

// 2. Search Handler Function
function handleSearch(query) {
  console.log(`[API Search Triggered]: "${query}" at ${new Date().toLocaleTimeString()}`);
}

// 3. Debounced Wrapper & Event Listener
const debouncedSearch = debounce((event) => {
  handleSearch(event.target.value.trim());
}, 500);

const searchInput = document.getElementById('search-input');
searchInput.addEventListener('input', debouncedSearch);
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** ব্যবহারকারী প্রতি ক্লিকে কিপ্রেস করলেও অপ্রয়োজনীয় ২০টি রিকোয়েস্ট না পাঠিয়ে টাইপিং থামার পর মাত্র ১টি নেটওয়ার্ক কল করবে, যা ব্যান্ডউইথ ও সার্ভার লোড বাঁচায়।

Lab #2 [BEGINNER] — Throttled Window Scroll Event Listener

সমস্যা ও লক্ষ্য: একটি scroll event-এ throttle ব্যবহার করুন যাতে প্রতি ৩০০ms-এ সর্বোচ্চ একবার স্ক্রল পজিশন গণনা ও কনসোলে লগ হয়।

SOLUTION IMPLEMENTATION (LAB #2)JAVASCRIPT

```
// 1. Timestamp-based Throttle Utility
function throttle(func, interval = 300) {
  let lastTime = 0;
  return function(...args) {
    const now = Date.now();
    if (now - lastTime ≥ interval) {
      lastTime = now;
      func.apply(this, args);
    }
  };
}

// 2. Scroll Progress Tracker
function logScrollPosition() {
  const scrollTop = window.scrollToY || document.documentElement.scrollTop;
  const docHeight = document.documentElement.scrollHeight - window.innerHeight;
  const progress = Math.round((scrollTop / docHeight) * 100);
  console.log(`[Throttled Scroll] Position: ${scrollTop}px | Read: ${progress}%`);
}

// 3. Throttled Listener Attachment with Passive Flag
window.addEventListener('scroll', throttle(logScrollPosition, 300), { passive: true });
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** ইউজার দ্রুত স্ক্রল করলে প্রতি সেকেন্ডে ৬০টির বেশি ইভেন্ট ফায়ার হয়। থ্রটলিংয়ের মাধ্যমে একে প্রতি সেকেন্ডে মাত্র ৩ বারে সীমাবদ্ধ রাখা হয়, যা লেআউট থ্র্যাশিং দূর করে।

Lab #3 [BEGINNER] — Native Responsive Image Lazy Loading

সমস্যা ও লক্ষ্য: একটি image-এ loading="lazy" ও srcset ব্যবহার করে আধুনিক নেটিভ লেজি লোডিং টেস্ট ও পরিমাপ করুন।

HTML DOM STRUCTUREHTML

```
<!-- High-Performance Native Responsive Lazy Image -->
<picture>
  <source srcset="hero-large.avif 1200w, hero-medium.avif 800w" type="image/avif">
  <source srcset="hero-large.webp 1200w, hero-medium.webp 800w" type="image/webp">
  
</picture>
```

SOLUTION IMPLEMENTATION (LAB #3)JAVASCRIPT

```
// Image Load Event Listener & Metrics
const lazyImg = document.querySelector('img.perf-lazy');

lazyImg.addEventListener('load', () => {
  console.log('✅ Image loaded successfully on viewport entry!');
  lazyImg.classList.add('loaded'); // CSS ফেইড-ইন ট্রানজিশন
});

lazyImg.addEventListener('error', () => {
  console.error('❌ Failed to load high-res image. Fallback triggered.');
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** decoding="async" ব্রাউজারের ডিকোডিং মেইন থ্রেড থেকে সরিয়ে নেয় এবং স্পষ্ট width/height থাকায় কোনো Layout Shift (CLS = 0) হয় না।

Lab #4 [BEGINNER] — Smooth 60FPS Animation with requestAnimationFrame

সমস্যা ও লক্ষ্য: requestAnimationFrame() ব্যবহার করে একটি box-কে স্ক্রিনে মসৃণভাবে মুভ করান এবং ডেল্টা টাইম অনুযায়ী ফ্রেম ড্রপ পরিহার করুন।

SOLUTION IMPLEMENTATION (LAB #4)

JAVASCRIPT

```
const box = document.getElementById('animated-box');
let startTimestamp = null;
const duration = 2000; // ২ সেকেন্ডে এনিমেশন শেষ হবে
const targetDistance = 400; // ৪০০ পিক্সেল ডানে সরবে

function animateBox(timestamp) {
  if (!startTimestamp) startTimestamp = timestamp;
  const elapsed = timestamp - startTimestamp;
  const progress = Math.min(elapsed / duration, 1); // ০ থেকে ১ পর্যন্ত প্রগ্রেস

  // Easing function (easeOutQuad)
  const easeProgress = 1 - (1 - progress) * (1 - progress);
  const currentX = easeProgress * targetDistance;

  // GPU-accelerated 3D Transform ব্যবহার (Reflow ও Repaint শূন্য)
  box.style.transform = `translate3d(${currentX}px, 0, 0)`;

  if (progress < 1) {
    requestAnimationFrame(animateBox); // পরবর্তী ফ্রেমের জন্য রিকোয়েস্ট
  } else {
    console.log('🎉 60FPS Smooth Animation Complete!');
  }
}

// এনিমেশন শুরু
requestAnimationFrame(animateBox);
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** setInterval এর বদলে rAF ব্রাউজারের 60Hz রিফ্রেশ হারের সাথে নিখুঁতভাবে টাইমিং মেলায় এবং ট্যাব ব্যাকগ্রাউন্ডে গেলে ব্যাটারি বাঁচিয়ে পজ করে।

Lab #5 [INTERMEDIATE] — High-Performance DOM Batching with DocumentFragment

সমস্যা ও লক্ষ্য: ১০০০টি DOM element তৈরি করুন এবং সাধারণ লুপের বদলে DocumentFragment ব্যবহার করে মাত্র ১টি Reflow ও Repaint-এ অ্যাপেন্ড করুন।

SOLUTION IMPLEMENTATION (LAB #5)

JAVASCRIPT

```
const container = document.getElementById('user-list');

// ❌ Bad Approach: প্রতি লুপে অ্যাপেন্ড (১০০০ বার রিফ্লো ও রিপেইন্ট)
// for (let i = 0; i < 1000; i++) container.appendChild(createNode(i));

// ✅ Master Production Approach: DocumentFragment ব্যবহার
console.time('DocumentFragment Insertion');

const fragment = document.createDocumentFragment(); // ভার্চুয়াল মেমোরি কন্টেইনার

for (let i = 1; i ≤ 1000; i++) {
  const item = document.createElement('li');
  item.className = 'list-item';
  item.textContent = `Customer Record #${i} - Verified Active`;
  fragment.appendChild(item); // মেমোরিতে নোড যুক্ত (কোনো DOM রিফ্লো নেই)
}

// মূল DOM-এ মাত্র ১টি ট্রানজেকশনে ১০০০টি নোড অ্যাপেন্ড
container.appendChild(fragment);

console.timeEnd('DocumentFragment Insertion');
console.log('✅ 1,000 DOM Nodes rendered in a single Composite tick!');
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** DocumentFragment কোনো মূল DOM নোড নয়, এটি ইন-মেমোরি র‍্যাপার। এটি DOM ট্রি-তে যুক্ত হওয়ার পর নিজেই অদৃশ্য হয়ে যায় এবং শুধু চাইল্ড নোডগুলো সংযুক্ত থাকে।

Lab #6 [INTERMEDIATE] — IntersectionObserver Image Lazy Loader with Skeleton Placeholder

সমস্যা ও লক্ষ্য: IntersectionObserver দিয়ে 'Loading...' স্কেলেটন থেকে একচুয়াল ইমেজ ভিউপোর্টে প্রবেশ করার পর লোড করুন ও আন-অবজার্ড করুন।

HTML DOM STRUCTURE

HTML

```

```

SOLUTION IMPLEMENTATION (LAB #6)

JAVASCRIPT

```
// 1. Observer Configuration
const observerOptions = {
  root: null, // ভিউপোর্ট ডিটেকশন
  rootMargin: '100px', // স্ক্রিনে আসার ১০০ পিক্সেল আগেই প্রি-লোড শুরু হবে
  threshold: 0.01 // ১% দৃশ্যমান হলেই ট্রিগার
};

// 2. IntersectionObserver Instance
const lazyObserver = new IntersectionObserver((entries, observer) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      const actualSrc = img.getAttribute('data-src');

      // Placeholder থেকে আসল সোর্সে রূপান্তর
      img.src = actualSrc;
      img.onload = () => {
        img.classList.remove('skeleton-loading');
        img.classList.add('img-visible');
      };

      // মেমোরি লিক রোধে টাস্ক শেষে আন-অবজার্ড
      observer.unobserve(img);
      console.log(`[LazyLoaded] ${actualSrc}`);
    }
  });
}, observerOptions);

// 3. Observe All Data-Src Images
document.querySelectorAll('img[data-src]').forEach(img => lazyObserver.observe(img));
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** স্কল ইভেন্ট লিসেনারের তুলনায় IntersectionObserver সম্পূর্ণ অ্যাসিনক্রোনাস এবং ব্রাউজারের কম্পোজিটর থ্রেডে চলে, ফলে মেইন থ্রেড ল্যাগ হয় না।

Lab #7 [INTERMEDIATE] — Dynamic import() for Module Lazy-Loading & Code Splitting

সমস্যা ও লক্ষ্য: Dynamic import() ব্যবহার করে ইউজার বাটনে ক্লিক করলে তবেই একটি ভারী গ্রাফ চাটিং মডিউল lazy-load করুন।

SOLUTION IMPLEMENTATION (LAB #7)

JAVASCRIPT

```
const chartBtn = document.getElementById('load-chart-btn');
const chartContainer = document.getElementById('chart-container');

chartBtn.addEventListener('click', async () => {
  try {
    chartBtn.disabled = true;
    chartBtn.textContent = 'Loading Chart Engine ... ';

    // ডাইনামিক ইমপোর্টের মাধ্যমে মডিউল শুধু প্রয়োজনে নেটওয়ার্ক থেকে নামবে
    const { renderSalesChart } = await import('./modules/heavyChart.js');

    const sampleData = [120, 190, 300, 500, 200, 350];
    renderSalesChart(chartContainer, sampleData);

    chartBtn.textContent = '✅ Chart Rendered';
  } catch (error) {
    console.error('Failed to dynamically load chart module:', error);
    chartBtn.textContent = '❌ Loading Failed';
    chartBtn.disabled = false;
  }
});
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** অ্যাপ্লিকেশন বুট হওয়ার সময় পুরো চাটিং লাইব্রেরির 800KB লোড না করে ইনিশিয়াল JS বান্ডেল হালকা রাখা যায়, যা LCP এবং TBT স্কোর উল্লেখযোগ্যভাবে বাড়ায়।

Lab #8 [INTERMEDIATE] — Debounced Search API with AbortController Stale Request Cancellation

সমস্যা ও লক্ষ্য: Typing -> Debounce -> fetch() -> AbortController আর্কিটেকচারে পুরাতন পেন্ডিং রিকোয়েস্ট বাতিল করার সম্পূর্ণ প্রোডাকশন সলিউশন।

SOLUTION IMPLEMENTATION (LAB #8)

JAVASCRIPT

```
let currentAbortController = null;

// Debounced Search with Abort Signal
const searchProducts = debounce(async (searchTerm) => {
  if (!searchTerm) return;

  // ১. পূর্বের রিকোয়েস্ট যদি এখনো পেন্ডিং থাকে, তাকে বাতিল করুন
  if (currentAbortController) {
    currentAbortController.abort();
    console.warn('[Request Aborted]: Canceled stale pending query');
  }

  // ২. নতুন রিকোয়েস্টের জন্য ফ্রেশ কন্ট্রোলার তৈরি
  currentAbortController = new AbortController();
  const { signal } = currentAbortController;

  try {
    const response = await fetch(`/api/products?search=${encodeURIComponent(searchTerm)}`, { signal });
    const data = await response.json();
    displayResults(data.results);
    console.log('[Search Results Loaded] Count: ${data.results.length}');
  } catch (err) {
    if (err.name === 'AbortError') {
      console.log('❗ Request successfully aborted as user typed new characters.');
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** নেটওয়ার্কের রেস কন্ডিশন (stale response overwriting latest result) সম্পূর্ণ দূর করে এবং অপ্রয়োজনীয় ব্যান্ডউইথ খরচ বন্ধ করে।

Lab #9 [ADVANCED] — Multi-Route SPA Dashboard Lazy-Loader with Component Caching

সমস্যা ও লক্ষ্য: Dashboard-এর Overview, Analytics, Reports, Settings ট্যাবসমূহ অন-ডিম্যান্ড lazy-load ও মডিউল ক্যাশ করার ক্লায়েন্ট-সাইড আর্কিটেকচার।

SOLUTION IMPLEMENTATION (LAB #9)

JAVASCRIPT

```
// Route-to-Module Manifest
const routeManifest = {
  overview: () => import('./views/overview.js'),
  analytics: () => import('./views/analytics.js'),
  reports: () => import('./views/reports.js'),
  settings: () => import('./views/settings.js')
};

// In-Memory Component Cache (একবার লোড হলে আর নেটওয়ার্ক কল হবে না)
const componentCache = new Map();
const contentArea = document.getElementById('dashboard-view');

async function navigateTab(routeKey) {
  if (!routeManifest[routeKey]) return;

  // ক্যাশ চেক
  if (componentCache.has(routeKey)) {
    console.log(`🔥 [Cache Hit] Rendering cached tab: ${routeKey}`);
    const cachedRender = componentCache.get(routeKey);
    cachedRender(contentArea);
    return;
  }

  // ক্যাশ মিস: নেটওয়ার্ক থেকে চাক্ষু লোড
  contentArea.innerHTML = '<div class="loader-spinner">Loading view module ... </div>';
  try {
    const module = await routeManifest[routeKey]();
    componentCache.set(routeKey, module.render); // ক্যাশে সেভ
    module.render(contentArea);
    console.log(`🌐 [Network Load] Module loaded & cached: ${routeKey}`);
  } catch (err) {
    contentArea.innerHTML = '<div class="error-msg">Failed to load view. Check connection.</div>';
  }
}

// Event Delegation on Tab Buttons
document.getElementById('tab-nav').addEventListener('click', (e) => {
  if (e.target.matches('button[data-route]')) {
    navigateTab(e.target.dataset.route);
  }
});
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** রুট-বেসড কোড স্প্লিটিং ইনিশিয়াল লোড সাইজ ৬০-৮০% কমিয়ে দেয় এবং ইন-মেমোরি ক্যাশিংয়ের কারণে একবার ভিজিট করা ট্যাবে ইন্সট্যান্ট সুইচিং ঘটে।

Lab #10 [ADVANCED] — Off-Main-Thread 100k JSON Data Processing with Web Worker

সমস্যা ও লক্ষ্য: ১ লক্ষ আইটেমের বিশাল JSON ডেটাসেট ফিল্টারিং ও সটিং করার সময় মেইন থ্রেড যাতে ১ মিলিমিটারও ফ্রিজ না হয় তার জন্য Web Worker সলিউশন।

SOLUTION IMPLEMENTATION (LAB #10)

JAVASCRIPT

```
// --- MAIN THREAD SCRIPT (app.js) ---
const worker = new Worker('./workers/dataProcessor.worker.js');
const statusBadge = document.getElementById('status-badge');

// ১. বিশাল ডেটাসেট প্রসেসিংয়ের জন্য ওয়ার্কারের কাছে পাঠানো
function processDataset(heavyData, filterKeyword) {
  statusBadge.textContent = 'Processing 100k items in Background Worker ...';
  console.time('Worker Processing Time');

  worker.postMessage({
    action: 'FILTER_AND_SORT',
    payload: heavyData,
    keyword: filterKeyword
  });
}

// ২. ওয়ার্কার থেকে ফলাফল রিসিভ
worker.onmessage = function(e) {
  const { sortedData, executionMs } = e.data;
  console.timeEnd('Worker Processing Time');
  statusBadge.textContent = `✅ Processed ${sortedData.length} records in ${executionMs}ms (Main thread stayed 60FPS!)`;
  renderTopResults(sortedData.slice(0, 50));
};

// --- WORKER SCRIPT (dataProcessor.worker.js) ---
// self.onmessage = function(e) {
//   const { payload, keyword } = e.data;
//   const t0 = performance.now();
//   const filtered = payload
//     .filter(item => item.name.toLowerCase().includes(keyword.toLowerCase()))
//     .sort((a, b) => b.score - a.score);
//   self.postMessage({ sortedData: filtered, executionMs: Math.round(performance.now() - t0) });
// };
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** মেইন থ্রেড মুক্ত থাকায় ইউজার ড্রপডাউন ওপেন করা, টাইপ করা বা স্ক্রল করার সময় কোনো লং-টাস্ক (>50ms) তৈরি হয় না এবং INP স্কোর চমৎকার থাকে।

Lab #11 [ADVANCED] — Programmatic Performance Profiling with User Timing API

সমস্যা ও লক্ষ্য: Chrome DevTools Performance প্যানেলে কাস্টম টাইমলাইন মেজারমেন্ট দেখতে `performance.mark()` ও `performance.measure()` দিয়ে কোড প্রোফাইলিং।

SOLUTION IMPLEMENTATION (LAB #11)

JAVASCRIPT

```
// ১. শুরু চিহ্নিত করা
performance.mark('data-fetch-start');

async function benchmarkDataFetch() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts');
  const data = await res.json();

  performance.mark('data-fetch-end');

  // ২. ফেটিংয়ের সময় পরিমাপ
  performance.measure('Data Fetch Duration', 'data-fetch-start', 'data-fetch-end');

  // ৩. DOM রেন্ডারিং পরিমাপ
  performance.mark('dom-render-start');
  const frag = document.createDocumentFragment();
  data.slice(0, 100).forEach(post => {
    const div = document.createElement('div');
    div.textContent = post.title;
    frag.appendChild(div);
  });
  document.getElementById('benchmark-container').appendChild(frag);
  performance.mark('dom-render-end');

  performance.measure('DOM Batch Rendering Duration', 'dom-render-start', 'dom-render-end');

  // ৪. মেজারমেন্ট ফলাফল কনসোলে বিশ্লেষণ
  const measures = performance.getEntriesByType('measure');
  measures.forEach(m => console.log(`🕒 [Metric] ${m.name}: ${m.duration.toFixed(2)} ms`));

  // মেমোরি পরিষ্কার
  performance.clearMarks();
  performance.clearMeasures();
}

benchmarkDataFetch();
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** User Timing API এর মাধ্যমে মেজারগুলো সরাসরি Chrome DevTools Performance ট্যাব-এর 'Timings' ট্র্যাকে দেখা যায়, যা বটলনেck চিহ্নিত করতে অত্যন্ত কার্যকর।

Lab #12

[ADVANCED] — End-to-End Production Performance Case Study & Audit Scorecard

সমস্যা ও লক্ষ্য: একটি ধীরগতির ইকমার্স প্রোডাক্ট পেজ অপ্টিমাইজেশনের বাস্তব কেস স্টাডি: Before vs After মেজারমেন্ট এবং স্পষ্ট অডিট রিপোর্ট।

SOLUTION IMPLEMENTATION (LAB #12)

JAVASCRIPT

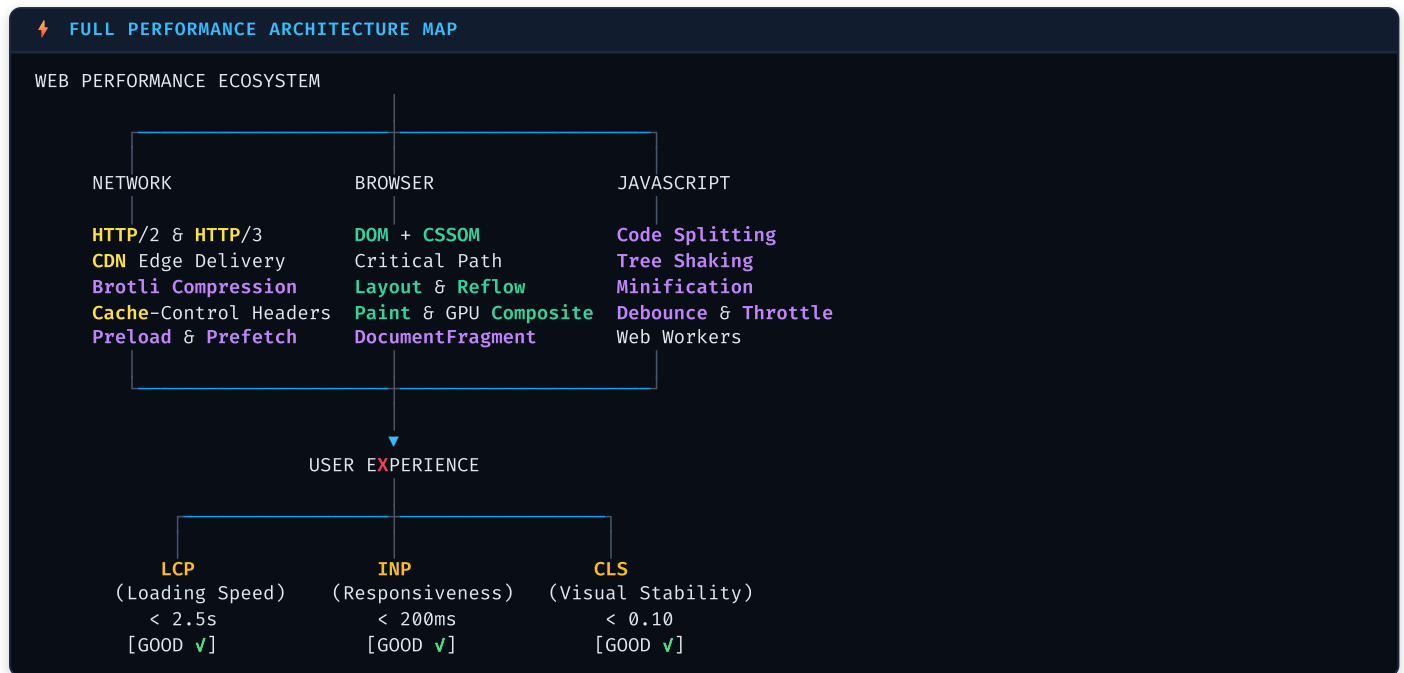
```
/*  
  
    REAL-WORLD E-COMMERCE PERFORMANCE CASE STUDY (BEFORE VS AFTER AUDIT)  
  
[PROBLEM AUDIT - BEFORE OPTIMIZATION]:  
* Initial JS Bundle: 1.8 MB (Monolithic bundle, zero code splitting)  
* Hero & Product Images: Uncompressed PNGs (Total 4.5 MB, no lazy loading)  
* Scroll Listener: Un-throttled scroll event recalculating layout on every tick  
* HTTP Cache: Missing Cache-Control headers, re-downloading static files on reload  
* Lighthouse Performance Score: 38 / 100  
* LCP (Largest Contentful Paint): 5.4 seconds (POOR ❌)  
* INP (Interaction to Next Paint): 480 ms (POOR ❌)  
* CLS (Cumulative Layout Shift): 0.38 (POOR ❌)  
  
[OPTIMIZATION INTERVENTIONS APPLIED]:  
1. Image Pipeline: Converted all PNGs to AVIF/WebP, added width/height & loading="lazy".  
2. Code Splitting: Split checkout & heavy charts via dynamic import() (Bundle reduced to 280 KB).  
3. Layout Thrashing Elimination: Replaced synchronous DOM reads with DocumentFragment & rAF.  
4. Network & Cache: Configured Cloudflare CDN with Cache-Control: max-age=31536000, immutable.  
5. Scroll Events: Added throttle(300ms) with { passive: true } on window scroll listeners.  
  
[VERIFICATION RESULTS - AFTER OPTIMIZATION]:  
* Initial JS Bundle: 280 KB (84% Size Reduction ✅)  
* Image Transfer Weight: 420 KB (90% Size Reduction ✅)  
* Lighthouse Performance Score: 98 / 100 (FAST 🚀)  
* LCP: 1.2 seconds (GOOD ✅ < 2.5s)  
* INP: 45 ms (GOOD ✅ < 200ms)  
* CLS: 0.02 (GOOD ✅ < 0.1)  
  
*/
```

💡 প্রোডাকশন অন্তর্দৃষ্টি:

প্রোডাকশনে পারফরম্যান্স অপ্টিমাইজেশন শুধু একটি টেকনিক নয়; এটি নেটওয়ার্ক, বান্ডলিং, ব্রাউজার রেন্ডারিং এবং ডেটাবেস ক্যাশিংয়ের একটি সামগ্রিক ইঞ্জিনিয়ারিং কালচার।

33.73 🧠 Final Mental Map — পারফরম্যান্স অপ্টিমাইজেশনের পূর্ণাঙ্গ মানচিত্র

পুরো Chapter 33-এর সকল মেকানিজম ও আর্কিটেকচারাল কানেকশন একসাথে:



🕒 এক লাইনে Chapter 33: Web Performance & Optimization হলো ব্রাউজারের নেটওয়ার্ক, ক্রিটিক্যাল রেন্ডারিং পাথ, জাভাস্ক্রিপ্ট বান্ডেল, মেমোরি ও ইভেন্ট এক্সিকিউশনকে এমনভাবে অপ্টিমাইজ করা যাতে অ্যাপ্লিকেশন চোখের পলকে লোড হয়, ফ্রেম ড্রপ ছাড়া রেসপন্স করে এবং কম রিসোর্সে সর্বোচ্চ ইউজার এক্সপেরিয়েন্স দেয়।

🔗 Curriculum Progression: Chapter 32 → 33 → 34

